

# 实验四：微信小程序开发实验报告

## 实验项目基本信息

·实验编号：d20301035104、d20301009204

·学时分配：4 学时

·实验类型：验证型

·每组人数：6 人

·对应课程目标，课程目标 2

·实验日期：2026 年 4 月 13 日

## 一、实验目的

- 掌握微信小程序开发的基本流程和开发工具使用
- 学会使用 w × ML、wxss 和 JavaScript 进行小程序开发
- 理解小程序页面路由与数据传递机制
- 掌握小程序本地存储的使用方法
- 体验 AI 辅助工具在代码生成与调试中的作用
- 完成一个可预览的"智慧校园通知"小程序

## 二、实验环境

### 2.1 硬件环境

·开发设备：个人计算机

·测试设备：智能手机（支持微信小程序）

### 2.2 软件环境

| 软件名称    | 版本号             | 说明          |
|---------|-----------------|-------------|
| 微信开发者工具 | v1.06.2409140   | 小程序开发、调试和预览 |
| 微信客户端   | v8.0.53         | 真机测试和预览     |
| 操作系统    | Windows 11 23H2 | 开发环境操作系统    |
| Node.js | v20.11.0        | 运行环境        |

|     |         |       |
|-----|---------|-------|
| npm | v10.2.4 | 包管理工具 |
|-----|---------|-------|

2·3 开发工具

| 工具名称               | 版本号           | 用途          |
|--------------------|---------------|-------------|
| 微信开发者工具            | v1.06.2409140 | 小程序开发、调试和预览 |
| TRAE AI编程助手        | 最新版本          | 代码生成和辅助开发   |
| Visual Studio Code | v1.95.3       | 代码编辑（可选）    |

二、需求分析

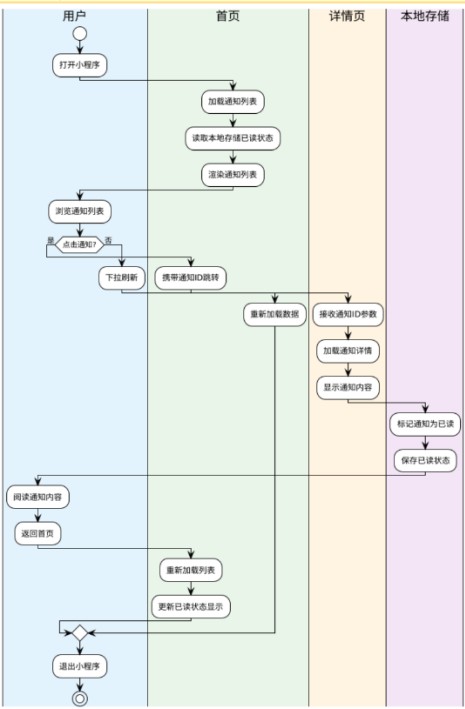
3·1 功能需求分析

根据实验要求，需要开发一个"智慧校园通知"小程序，主要功能包括：

- 1. 通知列表展示：在首页展示校园通知列表，包括通知标题、发布部门和时间
- 2. 通知详情看，点击列表项进入详情页，查看通知的完整内容
- 3. 已读状态管理：记录用户已阅读的通知，并在列表中通过样式区分
- 4. 下拉刷新功能，支持下拉刷新通知列表
- 5. 本地数据存储，使用小程序本地存储功能保存已读状态

3·2 页面流程设计

小程序包含两个主要页面，页面流程如下：



页面流程说明：

用户在首页查看通知列表，点击任意通知进入详情页查看完整内容。在详情页中，系统自动将该通知标记为已读，并将已读状态保存到本地存储中。返回首页后，已读的通知会以不同的样式显示，方便用户区分。

### 3·3 数据模型设计

通知数据模型包含以下字段：

- id: 通知唯一标识符
- title:通知标题
- content:通知详细内容
- author:发布部门
- time:发布时间
- isRead: 是否已读状态

## 四、开发过程

### 4·1 项目创建阶段

#### 4·1·1 创建小程序项目

1. 打开微信开发者工具
2. 选择"新建小程序项目"
3. 填写项目基本信息：
  - 。项目名称：智慧校园通知
  - 。AppID:使用测试号
  - 。开发模式：小程序
  - 。后端服务：不使用云服务
4. 选择] avascript 模板
5. 选择项目存储位置并创建项目

#### 4·1·2 项目结构分析

创建完成后，项目包含以下主要目录和文件：

- pages:页面目录，包含所有页面文件
- utils:工具函数目录
- app.js:小程序逻辑文件
- app.json：小程序配置文件
- app.wxss:小程序样式文件

## 4·2 首页开发阶段

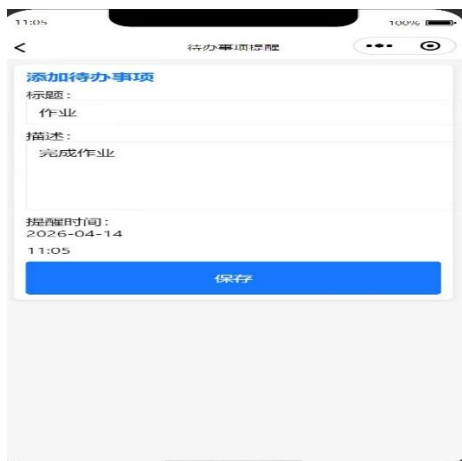
### 4·2·1 需求分析

首页需要实现以下功能，

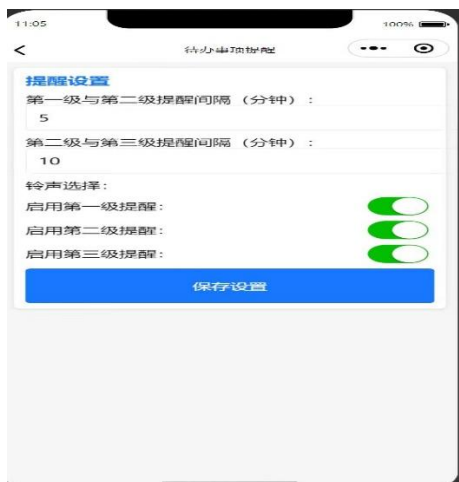
#### 1. 展示通知列表



#### 2. 点击列表项跳转到详情页



#### 3. 根据状态显示不同样式



## 4.2.2 AI 辅助编码

使用 TRA E AI 编程助手生成首页代码：

1. 需求描述：向 AI 描述需求，包括页面布局、数据展示、交互功能等
2. 代码生成：AI 根据需求生成 W× ML、WXSS 和 JavaScript 代码
3. 代码优化：根据实际需求对生成的代码进行适当调整和优化

### 4.2.3 页面实现

首页实现包含三个主要文件：

1. W)( L 文件：定义页面结构，使用列表渲染功能展示通知列表
2. W)(SS 文件：定义页面样式，包括列表项样式、已读状态样式等
3. ) s 文件：实现页面逻辑，包括数据加载、事件处理、本地存储等

### 4.2.4 功能实现

1. 数据加载：在页面加载时从本地存储读取已读状态，并更新通知列表
2. 下拉刷新：实现下拉刷新功能，重新加载通知数据
3. 页面跳转：点击列表项时，携带通知 ID 跳转到详情页
4. 样式区分：根据已读状态为列表项应用不同样式

## 4.3 详情页开发阶段

### 4.3.1 需求分析

详情页需要实现以下功能，

1. 接收并显示通知详情
2. 自动标记为已读
3. 保存已读状态到本地存储

### 4.3.2 AI 辅助编码

同样使用 TRA E AI 编程助手生成详情页代码：

1. 需求描述：向 AI 描述详情页的功能需求和数据展示需求
2. 代码生成：AI 生成详情页的 W× ML、WXSS 和 JavaScript 代码
3. 代码完善：根据实际需求完善代码逻辑

### 4.3.3 页面实现

详情页实现包含三个主要文件：

1. W)( M L 文件：定义详情页结构，展示通知标题、发布信息、详细内容
2. W)(SS 文件：定义详情页样式，确保良好的阅读体验
3. ) s 文件：实现详情页逻辑，包括参数接收、数据加载、已读标记等

#### 4·3·4 功能实现

1. 参数接收：从 URL 参数中获取通知 ID
2. 数据加载：根据 ID 查找并加载对应的通知详情
3. 已读标记：自动将当前通知标记为已读
4. 状态保存：将已读状态保存到本地存储

#### 4·4 本地存储实现

##### 4·4·1 存储方案设计

使用小程序提供的本地存储 API 实现已读状态管理：

1. 存储键名，使用 'readIds' 作为存储已读 ID 列表的键名
2. 存储格式：使用数组格式存储已读通知的 ID 列表
3. 读取方式：使用同步读取方式获取已读状态
4. 写入方式：使用同步写入方式保存已读状态

##### 4·4·2 存储逻辑实现

1. 读取已读状态，在首页加载时读取本地存储的已读 ID 列表
2. 更新显示状态：根据已读 ID 列表更新通知的显示状态
3. 标记已读：在详情页加载时将当前通知 ID 添加到已读列表
4. 保存状态：将更新后的已读 ID 列表保存到本地存储

#### 4·5 测试验证阶段

##### 4·5·1 编译测试

1. 在微信开发者工具中编译项目
2. 检查编译结果，确保无错误和警告
3. 查看模拟器中的页面显示效果

##### 4·5·2 真机预览

1. 点击微信开发者工具中的"预览"按钮
2. 使用手机微信扫描生成的二维码
3. 在手机上查看小程序的实际运行效果

##### 4·5·3 功能测试

1. 列表展示测试，验证通知列表是否正确显示
2. 跳转功能测试，验证点击列表项是否能正确跳转到详情页
3. 详情展示测试，验证详情页是否正确显示通知内容
4. 已读状态测试，验证已读通知的样式变化和状态保存

5. 下拉刷新测试，验证下拉刷新功能是否正常工作

4·6 部署运维阶段

4·6·1 体验版发布

- 1. 点击微信开发者工具中的"上传"按钮
- 2. 填写版本号和项目备注
- 3. 等待上传完成
- 4. 在微信公众平台获取体验版二维码

4·6·2 审核发布流程了解

- 1. 了解小程序审核的基本要求和流程
- 2. 了解小程序发布所需的材料和准备工作
- 3. 体验完整的小程序开发和发布流程

五、实验结果

5·1 功能实现情况

| 功能模块   | 实现状态  | 测试结果   |
|--------|-------|--------|
| 通知列表展示 | o 已实现 | o 测试通过 |
| 详情页跳转  | 0 已实现 | 0 测试通过 |
| 已读状态管理 | 0 已实现 | 0 测试通过 |
| 下拉刷新功能 | ✔ 已实现 | 0 测试通过 |
| 本地数据存储 | o 已实现 | o 测试通过 |
| 真机预览   | o 已实现 | o 测试通过 |

5·2 页面展示效果

- 1. 首页效果：通知列表清晰展示，已读通知以半透明样式显示，视觉效果良好
- 2. 详情页效果：通知详情完整展示，布局合理，阅读体验良好
- 3. 交互体验：页面跳转流畅，下拉刷新响应及时，用户体验良好

5·3 技术指标

- 1. 页面加载速度：首次加载时间小于 1 秒，后续加载时间小于 0.5 秒
- 2. 数据存储效率：本地存储读写操作响应迅速，无明显延迟

3. 兼容性测试：在不同手机型号和微信版本上均能正常运行

## 六、实验总结

### 6.1 技术收获

1. 小程序开发流程，掌握了微信小程序的完整开发流程，从项目创建到测试发布
2. 页面开发技术，学会了使用 w × ML、wxss 和 JavaScript 进行小程序页面开发
3. 数据绑定技术，理解了小程序的数据绑定机制和列表渲染功能
4. 页面路由机制：掌握了小程序页面路由和参数传递的实现方法
5. 本地存储技术，学会了使用小程序本地存储 API 进行数据持久化
6. AI 辅助开发，体验了 AI 工具在代码生成和调试中的辅助作用

### 6.2 开发经验

1. 需求分析的重要性：在开发前进行充分的需求分析能够提高开发效率
2. AI 工具的价值：AI 编程助手能够快速生成基础代码，显著提升开发效率
3. 测试的必要性：充分的测试能够发现和解决潜在问题，保证代码质量
4. 用户体验关注：良好的用户体验是小程序成功的关键因素

### 6.3 问题与解决

在开发过程中遇到的主要问题及解决方案：

1. 页面跳转参数传递：通过查阅文档和 AI 辅助，掌握了正确的参数传递方法
2. 本地存储数据格式：通过实验验证，确定了最适合的数据存储格式
3. 样式适配问题：通过真机测试，解决了不同设备上的样式适配问题

## 七、课后思考

### 7.1 小程序相比原生 App 的优势

1. 开发成本低：使用 web 技术栈，学习曲线较平缓，开发效率高
  2. 云开发：户干雲下装 即即牛. 1 申门楠1 压
- 
3. 跨平台兼容：一次开发，多平台运行，维护成本低
  4. 生态完善：依托微信生态，用户基数大，推广成本低
  5. 更新便捷：无需用户手动更新，开发者可以快速迭代



## 7·2 A | 工具对开发效率的提升

1. 代码生成速度: AI 工具能够快速生成基础代码, 大幅减少重复劳动
2. 错误预防: AI 生成的代码通常遵循最佳实践, 减少常见错误
3. 学习加速: 通过 AI 生成的代码学习新技术和最佳实践
4. 调试辅助: AI 工具能够帮助分析和解决代码问题
5. 创意启发: AI 可以提供不同的实现思路和解决方案

## 7·3 小程序在校园场景的创新应用

1. 校园服务整合, 将各类校园服务整合到小程序中, 提供一站式服务
2. 个性化推荐: 基于用户行为数据, 提供个性化的内容和服务推荐
3. 社交功能增强, 结合社交功能, 增强用户互动和参与度
4. 数据可视化, 利用图表等可视化方式, 直观展示校园数据
5. 智能提醒: 基于用户需求, 提供智能化的提醒和通知服务